

COMPLETE ECOSYSTEM REFERENCE V4.2

Shokunin 職人 AI Engineering Ecosystem

38 skills, ChromaDB persistent memory, 3 MCP servers, 4 subagents, cross-platform (Windows + Linux), multi-runtime (6 runtimes), CI/CD, zero cost, open source. The complete guide to building and using an AI engineering ecosystem that remembers.

Elias Oulkadi

Version 4.2 · May 2026 · 49 commits

github.com/EliasOulkadi/shokunin

| Contents

01 What is Shokunin?

02 Architecture & Five Layers

03 The 38 Skills

04 Memory System v4.2

05 MCP Servers

06 Subagents

07 Scripts & Automation

08 Cross-Platform (Windows + Linux)

09 Multi-Runtime (6 Runtimes)

10 CI/CD & Quality

11 Workflow Examples

12 Quick Reference

| What is Shokunin?

Shokunin (職人, Japanese for artisan) is an open source ecosystem that transforms AI coding agents into production-grade engineering workstations. [38 domain-specific skills](#), [persistent ChromaDB memory across sessions](#), and [cross-platform support](#) eliminate the blank-slate problem that plagues every AI session.

When you open a new AI coding session, the agent has zero context of your project, your decisions, or your codebase. Shokunin solves this by providing: domain expertise through auto-activating skills that teach the agent how to handle specific tasks; persistent memory through ChromaDB that saves context across sessions and retrieves it when you return; direct system access through MCP servers for filesystem, web fetching, and memory operations; and automated maintenance through platform-native scripting and scheduling.

Everything runs on your machine. No cloud dependencies, no telemetry, no subscriptions. Works on Windows and Linux, across six different AI coding runtimes (OpenCode, Claude Code, Cline, Cursor, Continue.dev, Windsurf). Zero cost, MIT license.

One command install:

Windows: `irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex`

Linux: `bash <(curl -sL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)`

| Architecture & Five Layers

The ecosystem is organized into five independent layers. Each layer serves a specific role and can be replaced or extended without affecting the others.

Layer 1: Interface

OpenCode, Claude Code, Cline, Cursor, Continue.dev, Windsurf. The AI coding runtime that the user interacts with. Shokunin adapts to each runtime through platform-specific configuration files in

`.pack/templates/` and rule files in `.pack/rules/`.

Layer 2: Agent

38 SKILL.md files, superpowers plugin, 4 subagents. Skills auto-activate based on task descriptions. Subagents handle code review and debugging in parallel. Instructions are enforced by CLAUDE.md and AGENTS.md.

Layer 3: Infrastructure

Three MCP servers (filesystem, fetch, memory) and ChromaDB vector database. The MCP servers start on demand via opencode.json and provide tools for file operations, web requests, and persistent memory. ChromaDB stores embeddings for semantic search.

Layer 4: AI Providers

NVIDIA Kimi K2.6 (free API), OpenAI, Anthropic, Ollama local (Qwen3 14B, DeepSeek, Qwen2.5). The ecosystem is provider-agnostic - change the model in opencode.json and everything works the same.

Layer 5: System

Platform-specific: Windows PowerShell profile, Task Scheduler, console buffer API; Linux bash/zsh profile, crontab, script command. This is the only platform-dependent layer. The core (skills, memory, MCP) is fully cross-platform.

Key Design Decisions

- **Capas independientes:** Cada capa se puede cambiar sin afectar las otras. Skills, memoria, scripts, configs estan separados.
- **Core cross-platform:** Python (chroma-helper.py, mcp-server.py) y SKILL.md funcionan en cualquier SO.
- **Fallback chain:** Cada operacion critica tiene 2-3 alternativas antes de fallar.
- **Cero regresiones:** Todos los cambios son aditivos. Nunca se modifica contenido existente.

Estadísticas del Ecosistema

Metrica	Valor
Commits	49
Skills	38 (cubriendo 9 dominios)
Scripts	17 (8 PowerShell + 6 bash + 3 Python)
PDFs documentacion	7
CI workflows	3 (validacion + memory tests + release)
Plataformas	Windows + Linux
Runtimes	OpenCode, Claude Code, Cline, Cursor, Continue.dev, Windsurf
Entradas ChromaDB	9 (y creciendo)
Bugs criticos arreglados	10

| The 38 Skills

Each skill is a SKILL.md file with YAML frontmatter, procedural workflow, error handling table, production checklist, anti-patterns, and cited sources. Skills activate automatically when you describe a task.

Skill Format

```

---
name: docker
description: Optimize Docker images with multi-stage builds...
license: MIT
compatibility: opencode
metadata:
  workflow: infrastructure
  audience: devops
  version: "3.0"
  author: shokunin
allowed-tools: Read Bash Write
---

# Docker Architect

Production-grade Dockerfiles, multi-stage builds, cache optimization...

## Workflow (numbered steps with decision tables)
## Error Handling (scenario/cause/fix table)
## Production Checklist (verifiable items)
## Anti-Patterns (what NOT to do)
## Sources (cited references)

```

Skills by Domain

Infrastructure (6)

Skill	Function	Trigger example
docker	Multi-stage builds, BuildKit, vulnerability scanning	"Containerize my app"
kubernetes	Gateway API, service mesh, HPA, PDB	"Deploy to K8s"
terraform	Stacks, test framework, state surgery	"Set up AWS infra"
ci-cd	GitHub Actions, GitLab CI, CircleCI	"Add CI pipeline"
db-admin	PostgreSQL backup, replication, PITR	"Backup database"
runbook-gen	Incident runbooks, post-mortems	"Site is down"

Backend (4)

Skill	Function	Trigger example
auth-architect	OAuth2, JWT, WebAuthn, RBAC, OWASP	"Add login"
api-forge	REST/GraphQL, OpenAPI 3.1, webhooks	"Design an API"
db-sculptor	Index strategy, EXPLAIN ANALYZE, Prisma	"Optimize this query"
error-handler	OpenTelemetry, retry, circuit breaker	"Add observability"

Frontend (5)

Skill	Function	Trigger example
component-forge	React/Vue/Svelte, WCAG 2.2	"Create a button component"
responsive-engine	Container Queries, clamp(), subgrid	"Make responsive"
motion-craft	WAAPI, Scroll-Driven Animations, FLIP	"Add animation"
landing-craft	CRO, LIFT Model, A/B testing	"Design a landing page"
ui-ux-pro-max	UI patterns DB, palettes, fonts	"Pick a color palette"

Mobile (2)

Skill	Function	Trigger example
flutter	Riverpod, Impeller, Clean Architecture	"Build Flutter app"
react-native	Expo Router, FlashList, Reanimated 4	"Build RN app"

Quality (2)

Skill	Function	Trigger example
test-commander	Testing Trophy, Vitest, MSW, Playwright	"Add tests"
performance-profiler	Lighthouse, Core Web Vitals	"Audit performance"

Content & Business (7)

Skill	Function	Trigger example
communication	Emails, SBI feedback, meeting notes	"Write an email"
content-marketing	AIDA/PAS/BAB, copywriting, Cialdini	"Write a blog post"
business-proposals	Cold emails, GBB pricing, SOWs	"Create a proposal"
seo-geo	SEO + GEO 2026, llms.txt, AI search	"Optimize for search"
translate-craft	8 idiomas, ICU, RTL	"Translate to Japanese"
documentation	READMEs, API docs, changelogs	"Write docs"
kami	Professional PDFs, parchment design	"Create a PDF"

Productivity (10)

Skill	Function	Trigger example
git-workflow	Branch/commit/PR automation	"Create branch"
windows-powershell	System info, cleanup, dev tools	"Check system health"
strategy	Brainstorming, RICE, pre-mortem	"Decide approach"
design	Brand guidelines, tokens, briefs	"Design system"
finance	5-pillar planning, tax, FIRE	"Plan finances"
legal-counsel	GDPR, AI Act, HIPAA, DMCA	"GDPR requirements"
whendone-plus	Desktop notifications for commands	"Notify when done"
memory	ChromaDB memory management	"Search memory"
chromadb	View, backup, reset vector DB	"Show memory stats"
cleanup-surgeon	File declutter, old files cleanup	"Clean up files"

Skill Quality

All 38 skills have been audited and upgraded. Each skill now includes: YAML frontmatter with trigger-optimized description, numbered procedural workflow, error handling table (scenario,cause,fix), production checklist (verifiable items), anti-patterns table (what NOT to do), cited sources (OWASP, NIST, official docs). Skills average 170 lines each, with the deepest (flutter: 405 lines, kami: 409 lines) being the most comprehensive.

Memory System v4.2

Three layers ensure no context is lost between sessions. ChromaDB provides semantic search; a parallel checkpoint timer saves every 5 minutes; the platform-specific wrapper captures terminal output on exit.

Layer 1: Agent-Driven Saves

The agent saves context during the session using `chroma-helper.py` directly via Bash/python tool. No MCP dependency. Every 3-5 interactions a checkpoint is saved. After each decision, file change, or command execution, the agent saves the context with type: decision, file, command, or checkpoint. The primary command is:

```
python ~/.shokunin/scripts/chroma-helper.py save "text" "session_id" "type" "tags" "project"
python ~/.shokunin/scripts/chroma-helper.py save "Decision: Use RS256 for JWT" "session-123"
"decision" "auth, jwt" "myapp"
```

Layer 2: Parallel Checkpoint Timer

Every 5 minutes, a background process saves a timestamped checkpoint. On Windows this is a .NET Timer thread. On Linux it is a bash sleep loop background process. This runs regardless of whether the agent saves anything. If the session crashes, at least the checkpoints remain.

Layer 3: Terminal Capture on Exit

When the session ends, the wrapper reads the terminal output. On Windows it reads the console buffer via `$host.UI.RawUI.GetBufferContents` (5000 lines, PS5.1 only). On Linux it uses the `script` command (unlimited lines). The output is stripped of ANSI codes and saved as both raw log (.log) and parsed markdown (.md).

Fallback Chain

1. **chroma-helper.py save** via Bash tool (primary, always works)
2. **store_context via MCP** (secondary, if MCP server is running)
3. **Markdown file** to `sessions/*.md` (fallback if both fail)
4. **current-session.json** written by wrapper (last resort metadata)

Data Format

```
{
  "type": "decision | file | command | checkpoint | session_end",
  "timestamp": "2026-05-14T10:30:00Z",
  "project": "my-project",
  "tags": ["auth", "jwt", "decision"],
  "session_id": "session-20260514-103000-A7F3",
  "text": "Descriptive context: what was done, why, and the result"
}
```

Storage

Location	Path	Size
ChromaDB vector database	~/.shokunin/memory/chroma_db/	400 KB + ~1 KB/entry
Session raw logs	~/.shokunin/memory/sessions/*.log	1-10 KB/session
Session summaries	~/.shokunin/memory/sessions/*.md	0.5-3 KB/session
Active session metadata	~/.shokunin/current-session.json	~200 bytes
Backups	~/.shokunin/backups/	Weekly zip

MCP Servers

MCP (Model Context Protocol) servers give the AI agent direct system access. They start on demand via `opencode.json`, consume no resources when idle, and work identically on Windows and Linux.

Server	Command	Tools	Configuration
filesystem	<code>npx @modelcontextprotocol/server-filesystem ~</code>	read, write, edit, search, list files	<code>opencode.json</code> , <code>claude-code.json</code> , <code>cline-settings.json</code>
fetch	<code>npx @modelcontextprotocol/server-fetch</code>	fetch web pages, <code>fetch_json</code> APIs	Same config files
memory	<code>python mcp-server.py</code>	<code>store_context</code> , <code>search_context</code> , <code>get_session_summary</code>	317 lines Python, JSON-RPC 2.0

Memory MCP Server (`mcp-server.py`)

The memory server is 317 lines of Python implementing a JSON-RPC 2.0 protocol over `stdin/stdout`. Three tools:

- **store_context:** Saves text with type, tags, project, `session_id`. Also writes to `sessions/*.md` as markdown backup.
- **search_context:** Semantic search via ChromaDB with optional project, type, and tag filters. Returns top 5 entries with similarity scores.
- **get_session_summary:** Retrieves all entries for a given session ID with a generated summary.

Key features: type field (decision, file, command, checkpoint, `session_end`, general), automatic markdown fallback, logging to `mcp-server.log`, error handling with Python `try/catch`.

| Subagents

Four specialized subagents run in parallel with their own models. They handle code review and debugging without blocking the main conversation.

Subagent	Primary Model	Fallback	Purpose
code-review	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Analyze diffs for bugs, security, code smells
code-review-local	Ollama Qwen3 14B	-	Offline code review (no internet)
debugger	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Root cause analysis from stack traces
debugger-local	Ollama Qwen3 14B	-	Offline debugging

Usage: "Review this diff for bugs" spawns the code-review subagent. "Debug this crash" spawns the debugger. The main session continues uninterrupted while the subagent works in the background.

Scripts & Automation

17 scripts form the backbone of the ecosystem. They handle session management, memory operations, health checks, file cleanup, and weekly maintenance.

Core Python

Script	Lines	Purpose
chroma-helper.py	76	Direct ChromaDB access: save, search, count. Cross-platform paths (USERPROFILE or HOME). Used by ALL other scripts.
mcp-server.py	317	JSON-RPC 2.0 MCP server for memory. store_context, search_context, get_session_summary.
read-transcript.py	65	Cross-platform transcript parser. Strips ANSI, extracts decisions/commands, outputs markdown.

Windows PowerShell (8)

Script	Lines	Purpose
run-opencode.ps1	192	Main wrapper: env vars, MCP healthcheck, buffer capture to 9999 lines, ChromaDB save, checkpoint timer
search-memory.ps1	83	Unified search: ChromaDB semantic + grep markdown + recent entries fallback. Never returns empty.
save-memory.ps1	57	Save with 2 retries, chroma-helper.py primary, markdown fallback
memory-healthcheck.ps1	131	8 tests: Python, chromadb, chroma-helper save/search, MCP tools, syntax, config, storage
test-memory.ps1	127	12 automated tests for memory system validation
scan-cleanup.ps1	170	File cleanup: scans Desktop, Downloads, TEMP. Classifies by metadata. Moves to Recycle Bin via Microsoft.VisualBasic.
read-transcript.ps1	72	ANSI stripping, decision/command extraction, markdown output
weekly-healthcheck.ps1	59	Disk check, memory backup, cleanup logs via Task Scheduler (Sunday 21:00)

Linux bash (6)

Script	Lines	Purpose
run-opencode.sh	105	Wrapper: script capture, checkpoint timer, ChromaDB save. No line limit on terminal capture.
memory-healthcheck.sh	53	Linux validation: Python, chromadb, save/search, MCP, configs, storage
test-memory.sh	60	Linux test suite: 5 sections, 10 checks
scan-cleanup.sh	55	Linux cleanup: TEMP files, shokunin PDFs, Desktop files
weekly-maintenance.sh	26	Crontab-ready: backup, cleanup, disk check (Sunday 21:00 via crontab)
profile.sh	41	bash/zsh aliases: git, npm, docker, opencode wrapper function

Cross-Platform: Windows & Linux

The same 38 skills, ChromaDB memory, MCP servers, and subagents work identically on both operating systems. Only the wrapper scripts, scheduler, and profile configuration are platform-specific.

Shared (identical)

Component	Windows	Linux
38 SKILL.md files	Yes	Yes
chroma-helper.py	Yes	Yes
mcp-server.py	Yes	Yes
CLAUDE.md	Yes	Yes
opencode.json	Yes	Yes
4 subagents	Yes	Yes
MCP servers	Yes	Yes

Platform-Specific

Capability	Windows	Linux Advantage
Terminal capture	Buffer API (5000 lines, PS5.1 only)	script command (unlimited)
Wrapper	run-opencode.ps1	run-opencode.sh
Installer	install.ps1 (467 lines, refactored from 517)	install.sh (206 lines)
Profile shadow	PowerShell \$PROFILE function	.bashrc/.zshrc function
Scheduling	Task Scheduler (Sunday 21:00)	crontab (Sunday 21:00)
Healthcheck	memory-healthcheck.ps1	memory-healthcheck.sh
Cleanup	scan-cleanup.ps1	scan-cleanup.sh
Tests	test-memory.ps1	test-memory.sh
Profile aliases	8 functions (gst, ga, gc, gp, ni, nrd, nrb, nt...)	.pack/scripts/profile.sh (bash/zsh)

The Linux version is not a second-class port. It has advantages like unlimited terminal capture via the `script` command (Windows is limited to 5000 lines in PS5.1). The shared Python core detects the OS automatically for path resolution (USERPROFILE on Windows, HOME on Linux).

| Multi-Runtime Support (6 Runtimes)

The ecosystem is not tied to OpenCode. Skills, memory, MCP servers, and scripts work across six different AI coding runtimes. This was a deliberate design decision to avoid vendor lock-in.

Runtime	Skills	Memory	MCP	Config File
OpenCode	Native	Native	opencode.json	Built-in
Claude Code	Reads SKILL.md	Via MCP	claude-code.json	.pack/templates/claude-code.json
Cline	Reads SKILL.md	Via .clinerules	cline-settings.json	.pack/templates/cline-settings.json
Cursor	Reads SKILL.md	Via .cursor/rules	cursor-mcp.json	.pack/templates/cursor-mcp.json
Continue.dev	Reads SKILL.md	Via .continue/rules	continue-config.yaml	.pack/templates/continue-config.yaml
Windsurf	Reads SKILL.md	Via .windsurf/rules	Via UI	.pack/rules/windsurf-memory.md

Template configs are in `.pack/templates/` (4 files) and runtime rules in `.pack/rules/` (4 files). No existing files are modified to add a runtime - everything is purely additive.

CI/CD & Quality

GitHub Actions validates every component on every push. Three workflows with four parallel jobs.

CI Workflows

Workflow	Jobs	Validates
ci.yml	validate	38 skills frontmatter, workflow, error handling, sources; JSON config; bash + Python syntax; CLAUDE.md.template integrity
memory-tests.yml	test-python, test-scripts, test-linux, validate-config	ChromaDB, MCP, PS1 parse, bash syntax, cross-platform paths
release.yml	release (on tag push)	Generates changelog from commits, builds archive, creates GitHub Release

Quality Standards

- **Senior Engineer Coding:** guard clauses, named constants, no magic numbers, edge case handling in every script
- **Error Handler:** retry with jitter (2 attempts), structured fallback chain, never silent catch
- **OWASP security:** no secrets in code, validated input (sed delimiter guard for API keys), principle of least privilege
- **Idempotent installer:** backup before write, skip existing, safe to re-run
- **Cero regresiones:** all changes are additive, never modifying existing content

10 Critical Bugs Found and Fixed

#	File	Bug	Fix
1	install.ps1	Set-Alias with arguments broken	Replaced with function definitions
2	run-opencode.ps1	Fallback wrote filename as content	Fixed to write summaryText variable
3	weekly-healthcheck.ps1	Disk percentage always 100%	Fixed formula: used/(used+free)*100
4	scan-cleanup.ps1	Shell.Namespace(o) was Desktop not Recycle Bin	Changed to Microsoft.VisualBasic.FileSystem.DeleteFile
5	memory-healthcheck.ps1/.sh	Overwriting current-session.json during test	Skipped destructive write
6	mcp-server.py	type filter parameter shadowed	Renamed variables (filter_type vs meta_type)
7	install.ps1	npx --version doesn't install MCP	Changed to npm install -g
8	profile.sh	Redefined touch (truncates files)	Removed dangerous function
9	memory-healthcheck.sh	Single quotes blocked variable expansion	Fixed grep quoting
10	run-opencode.ps1	Hardcoded path to opencode binary	Dynamic via Get-Command

Workflow Examples

Real scenarios showing the ecosystem in action. Each example shows which skills activate, what the agent does, and what gets saved to memory.

Building Authentication

User says: "Add Google OAuth login to my Express app"

1. **auth-architect** activates: generates OAuth2 + PKCE flow, JWT with RS256 rotation, session management
2. **db-sculptor** activates: creates User schema with Prisma, migration script, indexes
3. **error-handler** activates: wraps auth routes with structured error handling
4. **test-commander** activates: generates integration tests with mocked OAuth provider
5. **Memory:** saves "Added Google OAuth with PKCE flow, decision for RS256 signing" (type: decision)

Deploying to Production

User says: "Dockerize and deploy to Kubernetes"

1. **docker** activates: multi-stage Dockerfile with BuildKit cache, .dockerignore, compose for dev
2. **kubernetes** activates: Deployment, Service, Ingress, HPA, PDB manifests
3. **ci-cd** activates: GitHub Actions pipeline with build, test, deploy, healthcheck, rollback
4. **db-admin** activates: PostgreSQL backup strategy, connection pooling with PgBouncer
5. **Memory:** saves files modified (type: file), commands run (type: command)

Debugging an Incident

User says: "API returning 502 errors, investigate"

1. **runbook-gen** activates: starts incident template with severity assessment
2. **error-handler** activates: analyzes recent logs for 502 patterns, suggests root causes
3. **debugger** subagent: deep analysis of stack traces and error rates
4. **memory:** searches for similar incidents from past sessions (type: checkpoint)

Decluttering the System

User says: "Clean up old files I don't need"

1. **cleanup-surgeon** activates: runs scan-cleanup.ps1 (or .sh on Linux)
2. Scans Desktop, Downloads, TEMP for files older than thresholds
3. AI analyzes each file: classifies as safe (green), review (yellow), keep (red)
4. Presents summary: "12 files, 85 MB recoverable"
5. On confirmation, moves files to Recycle Bin via Microsoft.VisualBasic
6. **Memory:** "Cleanup: 8 files sent to Recycle Bin, 62 MB freed" (type: session_end)

Content Creation

User says: "Write a blog post about our launch and export as PDF"

1. **content-marketing** activates: AIDA framework, headline formula, SEO structure
2. **seo-geo** activates: structured data, llms.txt, AI Overview optimization
3. **kami** activates: parchment design, professional PDF generation
4. **Memory:** "Blog post created: launch announcement" (type: file)

Quick Reference

Key Paths

What	Path
Skills (38)	~/config/opencode/skills/
OpenCode config	~/config/opencode/opencode.json
CLAUDE.md	~/claude/CLAUDE.md
AGENTS.md	~/AGENTS.md
ChromaDB data	~/shokunin/memory/chroma_db/
Session logs	~/shokunin/memory/sessions/
Current session info	~/shokunin/current-session.json
MCP server log	~/shokunin/memory/mcp-server.log
Cleanup log	~/shokunin/logs/cleanup.log
Backups	~/shokunin/backups/
GitHub repo	github.com/EliasOulkadi/shokunin

Quick Commands

Action	Windows	Linux
Start AI session	opencode (auto-wrapper via profile)	opencode (auto-wrapper via profile)
Test memory system	.\test-memory.ps1	./test-memory.sh
Healthcheck	.\memory-healthcheck.ps1	./memory-healthcheck.sh
Search memory	search-memory.ps1 -Query "X"	python chroma-helper.py search "X"
Save to memory	save-memory.ps1 -Text "X"	python chroma-helper.py save "X" ...
Clean files	.\scan-cleanup.ps1 -Scan	./scan-cleanup.sh
Git status	gst	gst
Git commit	gc "message"	gc "message"
npm install	ni	ni
git add + commit	ga + gc "msg"	ga + gc "msg"
Profile reload	. \$PROFILE	source ~/.bashrc

Version History

Version	Date	Changes
v1.0	2025 Q4	Initial 29 skills
v2.0	2026 Q1	All skills rewritten with workflows, checklists, anti-patterns
v3.0	2026.04	12 skills upgraded, memory MCP server, subagents, superpowers plugin, Telegram bot
v4.0	2026.05.14	Memory rewrite: structured data, run-opencode.ps1, chroma-helper.py, 10 bugs fixed
v4.1	2026.05.14	Checkpoint timer, CI pipeline, profile shadow, 3 capture strategies, search-memory fix
v4.2	2026.05.14	Linux port, multi-runtime, 6 skills upgraded, 7 more skills upgraded, 10/10 quality push

Links

GitHub: github.com/EliasOulkadi/shokunin

Install Windows: `irm`

`https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex`

Install Linux: `bash <(curl -sL`

`https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)`

License: MIT · Zero cost · Open source

Version 4.2 · 49 commits · 38 skills · 17 scripts · 7 PDFs · 3 CI workflows